



Otto Ninja Pro Controller

Manual Técnico

Versión 2.0.0

ESP32 · HTTP REST

Web · Android

Capacitor 6

Documentación completa sobre arquitectura, protocolo de comunicación, funcionalidades y guía de despliegue para el controlador del robot humanode Otto Ninja.

Laboratorio de Robótica | 2025 | Uso Interno

Tabla de Contenidos

1. Introducción y Descripción General	3
1.1 Propósito del sistema	3
1.2 Alcance y audiencia objetivo	3
1.3 Tecnologías utilizadas	3
2. Arquitectura del Sistema	4
2.1 Diagrama de componentes	4
2.2 Estructura de archivos	4
2.3 Flujo de datos	5
3. Instalación y Configuración	5
3.1 Requisitos previos	5
3.2 Instalación local	5
3.3 Compilación para Android	6
3.4 Configuración del ESP32	6
4. Protocolo de Comunicación (API REST)	7
4.1 Convenciones generales	7
4.2 Endpoints disponibles	7
4.3 Sistema de debounce y throttling	9
5. Módulos Funcionales	10
5.1 Sistema de Conexión	10
5.2 Controles de Juego (Joystick y D-Pad)	10
5.3 Control de Brazos y Cabeza	11
5.4 Sistema de Ataques	11

5.5 Sistema de Sonido (Buzzer)	12
5.6 Sistema de Calibración	13
6. Interfaz de Usuario	14
6.1 Diseño y sistema de estilos	14
6.2 Navegación y paneles	14
6.3 Orientación y responsividad	15
7. Referencia de Código	15
7.1 Estado global (state)	15
7.2 Funciones principales	16
8. Solución de Problemas	17
9. Glosario	18

1. Introducción y Descripción General

1.1 Propósito del sistema

Otto Ninja Pro Controller es una aplicación web/móvil de control remoto en tiempo real diseñada para operar el robot humanode *Otto Ninja* equipado con un microcontrolador **ESP32**. Permite enviar comandos de movimiento, ejecutar secuencias de combate, reproducir efectos de sonido y ajustar la calibración mecánica de los servomotores, todo desde una interfaz táctil intuitiva optimizada para orientación horizontal.

1.2 Alcance y audiencia objetivo

Este manual está dirigido a:

- **Desarrolladores** que necesiten mantener, extender o depurar la aplicación.
- **Técnicos de laboratorio** que configuren el entorno de despliegue.
- **Instructores de robótica** que integren el controlador en actividades educativas.

1.3 Tecnologías utilizadas

FRONTEND

Vanilla JS + CSS

Sin frameworks. HTML5, CSS3 y JavaScript ES2020+ nativos.

EMPAQUETADO MÓVIL

Capacitor 6

Genera APK nativo Android a partir del código web.

HARDWARE OBJETIVO

ESP32

Microcontrolador con servidor HTTP REST embebido en la misma red WiFi.

BUILD ANDROID

Gradle 8 + JDK 17

Pipeline estándar Android Studio / CLI.

VERSIÓN

2.0.0

Versión estable con modo joystick, D-Pad y emoji icons.

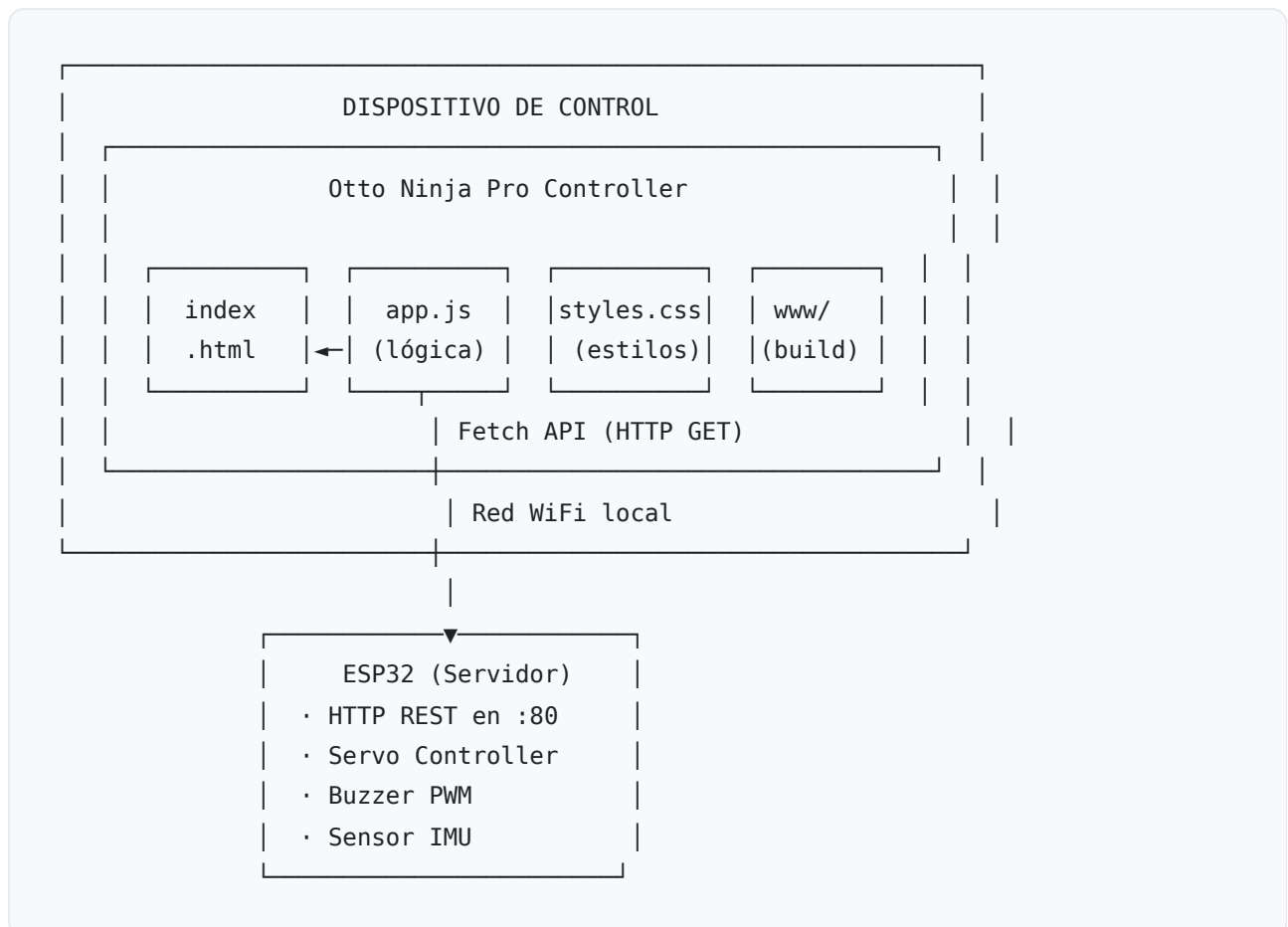
IDIOMA

Español

Interfaz completamente localizada al español.

2. Arquitectura del Sistema

2.1 Diagrama de componentes



La comunicación es **unidireccional**: la aplicación envía peticiones HTTP GET al ESP32, que ejecuta las acciones físicas y responde con un JSON simple de confirmación. No existe WebSocket ni subscripción de eventos; el estado del robot no se refleja en la UI.

2.2 Estructura de archivos

```

otto-ninja-controller/
├─ index.html           ← Estructura HTML de la interfaz (551 líneas)
├─ app.js               ← Toda la lógica de la aplicación (1 135 líneas)
├─ styles.css           ← Sistema de diseño y estilos (1 953 líneas)
├─ package.json         ← Metadatos y scripts npm
├─ capacitor.config.json ← Configuración Capacitor (Android)
├─ OTTO_NINJA.png       ← Logo del robot (38 KB)
├─ logo.png             ← Logo del laboratorio (83 KB)
├─ www/                 ← Assets web copiados para Capacitor
├─ android/             ← Proyecto Android Studio generado
│   └─ app/              ← Módulo principal Android
│       └─ capacitor-cordova-android-plugins/
│           └─ .gradle/

```

2.3 Flujo de datos



3. Instalación y Configuración

3.1 Requisitos previos

Componente	Versión mínima	Notas
Node.js	18 LTS	Necesario para scripts npm y Capacitor CLI
npm	9+	Incluido con Node.js
JDK	17	Requerido para compilar el APK Android
Android Studio	Iguana / 2023.2	O Gradle CLI ≥ 8.0
Navegador web	Chrome 120+ / Firefox 120+	Para uso como web app
Red WiFi	2.4 GHz o 5 GHz	El dispositivo y el ESP32 deben estar en la misma subred

3.2 Instalación y ejecución local

```
# 1. Clonar el repositorio
git clone https://github.com/Maykholl10/otto-ninja-controller.git
cd otto-ninja-controller

# 2. Instalar dependencias
npm install

# 3a. Servidor de desarrollo (con hot-reload)
npm run dev      # Abre http://localhost:8000

# 3b. Servidor de producción estático
npm start        # npx serve .
```

Nota sobre CORS

La aplicación envía peticiones HTTP a la IP del ESP32. Si se sirve desde `localhost`, los navegadores modernos requieren que el ESP32 incluya el header `Access-Control-Allow-Origin: *` en sus respuestas.

3.3 Compilación para Android (APK)

```
# Sincronizar assets web con el proyecto Android
npm run android:build
# Equivale a: npx cap sync android && npx cap open android

# Generar APK debug directamente desde CLI
npm run android:apk
# Equivale a: cd android && ./gradlew assembleDebug

# El APK resultante se encuentra en:
# android/app/build/outputs/apk/debug/app-debug.apk
```

⚠ Configuración de red en Android

El archivo `capacitor.config.json` establece el esquema HTTP (no HTTPS) para permitir peticiones a la IP local del ESP32 desde la APK Android. No modificar sin revisar las políticas de seguridad de red de Android.

3.4 Configuración del ESP32

El ESP32 debe estar programado con un firmware que:

- Se conecte a la red WiFi local y obtenga/use una IP estática conocida.
- Exponga un servidor HTTP en el puerto **80**.
- Responda a los endpoints definidos en la Sección 4.
- Incluya el header `Access-Control-Allow-Origin: *` en todas las respuestas.

✅ Conectar la app al robot

En la pantalla principal de la app, escribir la IP del ESP32 en el campo de texto (p. ej. `192.168.1.105`) y pulsar **Conectar**. La app verificará el endpoint `/status` para confirmar la conexión.

4. Protocolo de Comunicación (API REST)

4.1 Convenciones generales

- Todas las peticiones son **HTTP GET** a `http://<IP_ESP32>/<endpoint>`.
- Los parámetros se pasan en el **query string**: `?param=valor`.
- Las respuestas se esperan en **JSON**; cualquier respuesta no vacía se considera éxito.
- Timeout por petición: **2 000 ms** (controlado por `AbortController`).
- En caso de error de red, la función registra el error en el log pero **no reintenta**.

4.2 Endpoints disponibles

GET /status

Descripción: Verificación de conectividad (ping).

Parámetros: ninguno.

Respuesta esperada: cualquier respuesta JSON (p. ej. `{"ok":true}`).

Uso en app: función `connect()`.

GET /mode

Descripción: Cambia el modo de desplazamiento del robot.

Parámetros:

Parámetro	Valores	Descripción
cmd	rodar caminar	Modo joystick analógico o D-Pad

Ejemplo: `GET /mode?cmd=rodar`

GET /walk

Descripción: Ordena al robot caminar en una dirección.

Parámetro	Valores	Descripción
cmd	forward backward left right stop	Dirección de marcha

Ejemplo: GET /walk?cmd=forward

GET /joystick

Descripción: Envía la posición analógica del joystick (modo Rodar).

Parámetro	Rango	Descripción
x	-100 ... 100	Eje horizontal (negativo = izquierda)
y	-100 ... 100	Eje vertical (negativo = adelante)

Ejemplo: GET /joystick?x=45&y=-78

Frecuencia: máximo cada 100 ms (throttling en cliente).

GET /arms

Descripción: Controla los servos de los brazos.

Parámetro	Valores	Descripción
cmd	up down left right home	Posición de los brazos

GET /head

Descripción: Gira la cabeza del robot.

Parámetro	Valores	Descripción
cmd	left right center	Dirección del giro de cabeza

GET /attack

Descripción: Ejecuta una secuencia de combate preprogramada.

Parámetro	Valores	Descripción
cmd	slash uppercut spin360 estocada defensa combo_furia combo_samurai burla ultimate	Identificador del ataque

Ejemplo: GET /attack?cmd=slash

GET /buzzer

Descripción: Reproduce un efecto de sonido o melodía.

Parámetro	Rango	Descripción
song	1 ... 16	Número de canción/efecto pregrabado

Ejemplo: GET /buzzer?song=3

GET /offset

Descripción: Ajusta el offset de calibración de los servos de las piernas.

Parámetro	Rango	Descripción
left	-90 ... 90	Desplazamiento angular del servo izquierdo (grados)
right	-90 ... 90	Desplazamiento angular del servo derecho (grados)

Ejemplo (ambas piernas): GET /offset?left=5&right=-3

Ejemplo (solo izquierda): GET /offset?left=5

4.3 Sistema de debounce y throttling

Para evitar la saturación del ESP32, la aplicación implementa dos mecanismos de control de frecuencia de comandos:

Mecanismo	Intervalo	Aplica a
Cooldown global	400 ms	Todos los comandos de acción (walk, arms, head, attack, buzzer)
Throttle de joystick	100 ms	Envío continuo de posición X/Y del joystick analógico
Deduplicación	—	No envía el mismo comando walk si ya estaba activo

```
// Fragmento del sistema de debounce (app.js)
function canSendCommand(commandType) {
  const now = Date.now();
  const lastTime = state.lastCommandTime[commandType] || 0;
  if (now - lastTime < 400) return false;
  state.lastCommandTime[commandType] = now;
  return true;
}
```

5. Módulos Funcionales

5.1 Sistema de Conexión

La función `connect()` (*app.js:275*) gestiona la conexión al ESP32:

1. Lee la IP del campo de texto del header.
2. Valida que no esté vacía.
3. Realiza una petición a `/status` con `sendRequest()`.
4. Si responde correctamente: actualiza el indicador visual a verde, guarda la IP en `state.esp32IP` y habilita todos los controles.
5. Si falla: muestra estado "Desconectado" en rojo.

Persistencia de IP

La IP se almacena en `localStorage` bajo la clave `esp32IP`, de modo que se restaura automáticamente al recargar la aplicación.

5.2 Controles de Juego: Joystick y D-Pad

El panel de controles tiene **dos modos** seleccionables mediante tabs:

Modo Rodar (Joystick Analógico)

Implementado en *app.js:713-937*. Utiliza la **Pointer Events API** para capturar input táctil y de ratón de forma unificada.

- **Radio máximo:** 100 px desde el centro.
- **Normalización:** Los valores X/Y se normalizan a rango `[-100, 100]`.
- **Frecuencia de envío:** máximo 10 actualizaciones/segundo (100 ms).
- **Pointer capture:** se usa `setPointerCapture()` para mantener el arrastre fuera del elemento.

```
// Ciclo de actualización del joystick (simplificado)
function sendJoystickUpdate() {
  const now = Date.now();
  if (now - state.joystick.lastUpdate < 100) return;
  state.joystick.lastUpdate = now;

  const x = Math.round(state.joystick.x);
  const y = Math.round(state.joystick.y);
  sendRequest('joystick', { x, y });
}
```

Modo Caminar (D-Pad)

Botones direccionales clásicos:

Botón	Comando	Descripción
↑ Adelante	/walk?cmd=forward	Avanza hacia adelante
↓ Atrás	/walk?cmd=backward	Retrocede
← Izquierda	/walk?cmd=left	Gira/desplaza a la izquierda
→ Derecha	/walk?cmd=right	Gira/desplaza a la derecha
■ Home	/walk?cmd=stop	Detiene el movimiento

5.3 Control de Brazos y Cabeza

Panel *Brazos* (*app.js:341-373*). Dos grupos de botones independientes:

Brazos

Botón	Endpoint	Acción
Brazos Arriba	/arms?cmd=up	Eleva ambos brazos
Brazos Abajo	/arms?cmd=down	Baja ambos brazos
Brazo Izq.	/arms?cmd=left	Mueve brazo izquierdo
Brazo Der.	/arms?cmd=right	Mueve brazo derecho
Posición Home	/arms?cmd=home	Posición neutral

Cabeza

Botón	Endpoint	Acción
Girar Izquierda	/head?cmd=left	Gira la cabeza a la izquierda
Centrar	/head?cmd=center	Cabeza al frente
Girar Derecha	/head?cmd=right	Gira la cabeza a la derecha

5.4 Sistema de Ataques

Panel *Ataques* (*app.js:375-519*). El usuario selecciona hasta **4 ataques favoritos** de un catálogo de 9. Los seleccionados aparecen como botones de acceso rápido en el panel de controles.

ID	Nombre	Icono	Descripción
slash	Slash		Golpe lateral rápido
uppercut	Uppercut		Gancho ascendente
spin360	Giro 360°		Rotación completa
estocada	Estocada		Golpe frontal preciso
defensa	Defensa		Posición defensiva
combo_furia	Combo Furia		Combo agresivo multi-golpe
combo_samurai	Combo Samurai		Secuencia estilo samurai
burla	Burla		Animación de provocación
ultimate	Ultimate		Secuencia especial completa

⚠ Restricción de selección

Solo se pueden seleccionar **máximo 4 ataques**. Al intentar seleccionar un quinto, la app muestra un aviso y deshabilita el checkbox. El contador *X/4* en la UI refleja el número actual de selecciones.

5.5 Sistema de Sonido (Buzzer)

Panel *Sonido* (*app.js:521-654*). Catálogo de **16 efectos de sonido** numerados del 1 al 16. Igual que los ataques, el usuario pre-selecciona hasta **4 sonidos favoritos**.

#	Nombre	Icono	#	Nombre	Icono
1	Conexión		9	Feliz	
2	Desconexión		10	Triste	
3	Inicio batalla		11	Sorprendido	
4	Victoria		12	Enojado	
5	Derrota		13	Confundido	
6	Ataque		14	Asustado	
7	Defensa		15	Risa	
8	Power Up		16	Melodía especial	

Al reproducir un sonido, la UI muestra un **visualizador animado** con barras que simulan el espectro de audio. La animación dura aproximadamente 3 segundos (tiempo estimado de reproducción).

5.6 Sistema de Calibración

Panel *Calibrar* (*app.js:656-711*). Permite ajustar mecánicamente la posición neutral de las piernas mediante sliders de offset angular.

Control	Parámetro	Rango	Descripción
Slider pierna izquierda	left	-90° ... +90°	Offset del servo izquierdo
Slider pierna derecha	right	-90° ... +90°	Offset del servo derecho

Botones de acción de calibración

Botón	Endpoint generado	Descripción
Aplicar Izquierda	/offset?left=N	Aplica solo el offset izquierdo
Aplicar Derecha	/offset?right=N	Aplica solo el offset derecho
Aplicar Ambos	/offset?left=N&right=M	Aplica ambos offsets simultáneamente
Resetear	/offset?left=0&right=0	Vuelve a calibración cero

6. Interfaz de Usuario

6.1 Diseño y sistema de estilos

El sistema de diseño está definido enteramente en `styles.css` (1 953 líneas) y sigue principios del diseño de Apple HIG adaptados a robótica.

Token	Valor	Uso
Color primario	#0066CC	Botones, acentos, links
Color éxito	#34C759	Estado conectado, confirmaciones
Color peligro	#FF3B30	Errores, desconexión, ultimate
Color advertencia	#FF9500	Límites de selección
Fuente base	-apple-system, Segoe UI	Todo el texto
Radio mínimo botón	44 px	Accesibilidad táctil (WCAG)
Sombra base	0 2px 8px rgba(0,0,0,.12)	Cards, paneles

6.2 Navegación y paneles

La app utiliza un **Navigation Drawer** lateral (hamburger menu) que muestra 5 secciones. Al seleccionar una, la función `showPanel(panelId)` (*app.js:243*) oculta todos los paneles y muestra el solicitado mediante la clase CSS `.active`.

Panel ID	Nombre en UI	Icono SVG
game-controls	Controles	Gamepad
arms	Brazos	Robot arm
attacks	Ataques	Lightning bolt
buzzer	Sonido	Music note
calibrate	Calibrar	Settings sliders

6.3 Orientación y responsividad

- La app está optimizada para **orientación horizontal (landscape)**.
- El meta tag `viewport` y la configuración Capacitor fuerzan landscape en Android.
- Ancho máximo del contenido: **1 200 px** (centrado en pantallas grandes).
- Los botones tienen un mínimo de 44×44 px para facilitar el toque en pantallas pequeñas.
- Los grid layouts se colapsan a 1 columna en pantallas muy estrechas.

7. Referencia de Código

7.1 Estado global (`state`)

Toda la información de la sesión se mantiene en el objeto `state` definido al inicio de `app.js`:

```
const state = {
  // Conexión
  esp32IP:      '',          // IP del ESP32 activo
  isConnected:  false,

  // Navegación
  currentPanel: 'game-controls',
  gameMode:     'rotate',    // 'rotate' | 'walk'

  // Joystick
  joystick: {
    isDragging: false,
    x: 0, y: 0,          // Posición normalizada [-100, 100]
    lastUpdate: 0        // Timestamp del último envío
  },

  // Selecciones
  selectedAttacks: [],      // IDs de ataques (max 4)
  selectedSounds: [],      // IDs de sonidos (max 4)

  // Anti-rebote
  lastCommandTime: {},     // Mapa commandType → timestamp
  lastWalkCmd:     null    // Último comando walk enviado
};
```

7.2 Funciones principales

Función	Línea	Descripción
<code>sendRequest(endpoint, params)</code>	131	Realiza la petición HTTP GET al ESP32 con timeout de 2 s
<code>addLog(message)</code>	112	Registra eventos en el log interno con timestamp
<code>connect()</code>	275	Establece/verifica conexión con el ESP32
<code>showPanel(panelId)</code>	243	Cambia el panel visible en la UI
<code>switchGameMode(mode)</code>	182	Alterna entre modo joystick y D-Pad
<code>walk(cmd)</code>	324	Envía comando de desplazamiento con deduplicación
<code>arm(cmd)</code>	341	Controla los brazos del robot
<code>headMove(cmd)</code>	361	Mueve la cabeza del robot
<code>attack(cmd)</code>	491	Ejecuta ataque si está seleccionado y fuera del cooldown
<code>playSong(songNumber)</code>	632	Reproduce sonido con animación de visualizador
<code>toggleAttackSelection(id, cb)</code>	382	Gestiona la selección/deselección de ataques (límite 4)
<code>toggleSoundSelection(id, cb)</code>	528	Gestiona la selección/deselección de sonidos (límite 4)
<code>updateOffsetDisplay(leg, value)</code>	663	Actualiza el display de grados en calibración
<code>applyOffset(leg)</code>	680	Envía el offset de calibración al ESP32
<code>handleJoystick(event)</code>	784	Procesa eventos de puntero del joystick
<code>updateJoystickVisual(dx, dy)</code>	720	Actualiza la posición visual del thumb del joystick
<code>sendJoystickUpdate()</code>	736	Throttle y envío de datos del joystick al ESP32
<code>init()</code>	1080	Inicialización de la app: event listeners, estado, localStorage

8. Solución de Problemas

Síntoma	Causa probable	Solución
El botón "Conectar" queda en rojo	IP incorrecta, ESP32 apagado o fuera de red	Verificar que el ESP32 esté encendido y conectado al mismo WiFi. Comprobar la IP en el router.
Los comandos no llegan al robot	Error CORS o firewall	Confirmar que el firmware del ESP32 incluye el header Access-Control-Allow-Origin: *.
El joystick no responde en móvil	Touch events no capturados	Verificar que el navegador soporte Pointer Events API. Usar Chrome Mobile ≥ 100 .
Los ataques no se ejecutan	Ataque no seleccionado o cooldown activo	Seleccionar el ataque en el panel <i>Ataques</i> y esperar 400 ms entre comandos.
El robot se mueve de forma incorrecta en modo walk	Servos descalibrados	Usar el panel <i>Calibrar</i> para ajustar los offsets de cada pierna.
No se escuchan sonidos	Número de canción fuera de rango o buzzer deshabilitado en firmware	Verificar que el firmware soporte el número de canción seleccionado (1-16).
La APK Android no conecta al ESP32	Política de red HTTP bloqueada	Verificar que capacitor.config.json tenga "scheme": "http" y que la APK esté instalada con permisos de red local.
Gradlew falla en la compilación	JDK incompatible o SDK de Android desactualizado	Usar JDK 17. Ejecutar <code>npx cap sync android</code> antes de compilar.
La interfaz muestra orientación incorrecta	Falta de soporte landscape en el dispositivo	Forzar rotación landscape en configuración del dispositivo o verificar el meta tag viewport.

● **Advertencia de seguridad**

La API REST del ESP32 no implementa autenticación. **No exponer la red WiFi del robot a internet.** Usar exclusivamente en redes locales cerradas (laboratorio, evento).

9. Glosario

Término	Definición
APK	Android Package – archivo de instalación de aplicación para dispositivos Android.
Buzzer	Dispositivo piezoeléctrico del robot que genera tonos PWM; se controla con el endpoint /buzzer.
Capacitor	Framework de Ionic que envuelve una web app en un contenedor nativo Android/iOS.
CORS	<i>Cross-Origin Resource Sharing</i> – política del navegador que requiere que el servidor indique explícitamente qué orígenes pueden acceder a sus recursos.
D-Pad	<i>Directional Pad</i> – botones digitales de dirección (arriba/abajo/izq/der), como en mandos de consola.
Debounce / Cooldown	Técnica para limitar la frecuencia de envío de comandos, evitando saturar el receptor.
ESP32	Microcontrolador de Espressif con WiFi integrado; actúa como cerebro y servidor HTTP del robot Otto Ninja.
Gradle	Sistema de construcción de proyectos Android; compila el código fuente y genera el APK.
HIG	<i>Human Interface Guidelines</i> – guía de diseño de interfaces de Apple.
Joystick analógico	Control de entrada que reporta posición X/Y continua, como un mando de PS5.
Offset de calibración	Valor angular en grados que se suma al ángulo de reposo de un servo para compensar imprecisiones mecánicas del montaje.
Otto Ninja	Variante del robot educativo Otto con temática ninja; versión de combate con brazos articulados y cabeza giratoria.
Pointer Events API	API de navegador unificada para mouse, touch y stylus; permite un único handler para desktop y móvil.
PWM	<i>Pulse Width Modulation</i> – técnica para controlar servos y el buzzer mediante señales de ancho de pulso variable.
Query string	Parte de una URL tras el carácter ? que contiene parámetros clave=valor.

Término	Definición
REST API	<i>Representational State Transfer</i> – estilo de API que usa endpoints HTTP para operaciones CRUD.
Servo	Motor de precisión controlado por PWM que gira a un ángulo específico; utilizado en articulaciones del robot.
Throttling	Limitación de la tasa de eventos; similar al debounce pero permite eventos periódicos en lugar de solo el último.
www/	Directorio de assets web compilados que Capacitor copia al proyecto Android.

Otto Ninja Pro Controller · Manual Técnico v2.0.0

Laboratorio de Robótica · Documento de uso interno · 2025

app.js · index.html · styles.css · capacitor.config.json · package.json